

N Northeastern University
Electrical and Computer Engineering
EECE 6544: Introduction to Machine Learning and Pattern Recognition

Summer 2026

Assignment #05

Assignment: Hyperparameter Tuning a Neural Network

Dataset: diabetes.csv (768 patients, 7 clinical features, binary label: positive/negative) **Starter code:** train_diabetes.py (provided) **Estimated time:** 3–5 hours

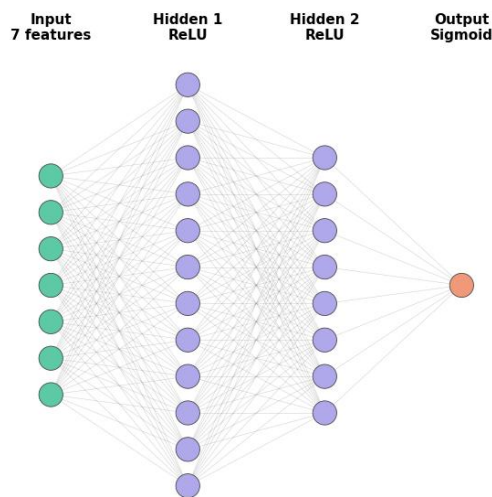
Background

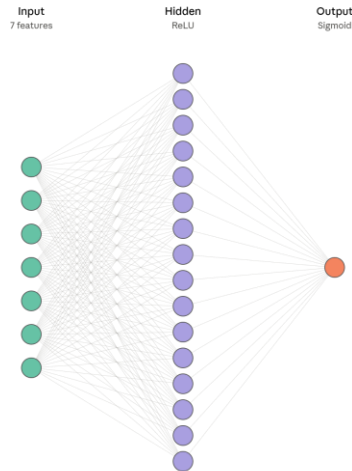
The starter code trains a small neural network that predicts whether a patient has diabetes. It reaches roughly 76% accuracy but every number in that script (learning rate, batch size, number of epochs, layer sizes) was chosen arbitrarily. These values are called **hyperparameters**: settings you choose *before* training, as opposed to weights, which the model learns *during* training.

Your job is to investigate how these choices affect model performance, and to find a better configuration than the default.

Part 1: Searching combinations

The diagram below shows a fully connected neural network. Write the Python code that defines this architecture using PyTorch. Your code must match the diagram exactly: the number of neurons in each layer and the activation function after each layer.





Part 2: Hyperparameters

1. List every hyperparameter in the case-study code, and explain why the weights inside `nn.Linear(7, 12)` are not hyperparameters.

Hyperparameter	Value here	Role	Risk if badly chosen
Learning rate η	0.001	step size of each update	diverges (big) / stalls (small)
Batch size	10	samples per update	noisy (small) / slow, smooth (large)
Epochs	100	passes over the data	underfits (few) / overfits (many)
Architecture	7→12→8→1	model capacity	too weak / memorizes
Activations	ReLU, Sigmoid	nonlinearity, output range	
Loss, optimizer	BCE, Adam	error measure, update formula	

Why the weights inside `nn.Linear(7, 12)` are *not* hyperparameters: they're **learnable parameters**. They start as random values and are changed *by* the training loop itself every `optimizer.step()` call updates them using gradients from `loss.backward()`.

2. The code **uses $\eta = 0.001$** . Predict what the printed **loss would do with** (a) $\eta = 5.0$ and (b) $\eta = 10^{-7}$, and justify using $\mathbf{w} \leftarrow \mathbf{w} - \eta \partial E / \partial \mathbf{w}$.
3. With 768 patients and batch size 10: (a) how many weight updates per epoch? (b) over 100 epochs? (c) what is special about the last batch of each epoch?
4. Keeping everything else fixed, what changes in the procedure if the batch size is set to (a) 768 and (b) 1? Name each variant of gradient descent.

5. The code always runs exactly 100 epochs. What risk does that fixed choice carry, and what measurement would the code need in order to stop at the right time?
6. Give two reasons the output activation must be a sigmoid here, referencing the targets built in step 1 and the loss chosen in step 3.

Part 3: Mathematics

7. Show, layer by layer, that the network has exactly 209 learnable parameters.
8. A hidden neuron has weights $w = [0.1, -0.2]$ on two inputs, bias $b = 0.3$, and receives $x_1 = 0.5$, $x_2 = 1.0$. (a) Compute z . (b) Compute the ReLU output. (c) Repeat with $b = -0.3$ and state what happens to the gradient through this neuron.
9. With $\sigma(z) = 1/(1+e^{-z})$: (a) compute $\sigma(0)$; (b) compute $\sigma(0.15)$ to three decimals; (c) what does σ approach as $z \rightarrow \pm\infty$, and why does that suit a probability?
10. A patient's true label is $t = 1$. Compute the BCE loss for (a) $\hat{y} = 0.9$ and (b) $\hat{y} = 0.1$. What principle does the comparison demonstrate?
11. One weight has $w = 0.37$, the learning rate is $\eta = 1.2$, and backpropagation finds $\partial E / \partial w = -0.0201$. (a) Apply the update rule. (b) Explain in one sentence why the weight increased despite the minus sign.
12. For the output neuron, E depends on w through $\hat{y} = \sigma(z)$ and $z = \sum w x + b$. (a) Write $\partial E / \partial w$ as a product of three factors. (b) Which factor equals just the input x , and why? (c) Given $\sigma'(z) = \sigma(z)(1-\sigma(z))$, what is the largest value σ' can take, and at which z ?
13. A student deletes `optimizer.zero_grad()`. Write what each weight's stored gradient contains after batch 3 (in terms of the per-batch gradients g_1, g_2, g_3) and state why the resulting updates are wrong.
14. Explain what breaks mathematically if `optimizer.step()` were called before `loss.backward()` in the first iteration, and why all gradients must be computed before any weight moves.